

markpublish Benutzerhandbuch

Moderne PDF- und HTML-Dokumentenerstellung aus Markdown

Dieses Handbuch bietet eine praxisorientierte Anleitung und vollständige Referenz zur Konfiguration, Template-Anpassung und Dokumentengenerierung mit markpublish.

Version	1.0.0
Datum	02.09.2026
Autor	Frank Winter
Copyright	© 2026 Frank Winter

Inhaltsverzeichnis

Die CLI-Befehle	4
Befehlsübersicht	4
<code>markpublish init</code>	4
<code>markpublish cheatsheet</code>	4
<code>markpublish manual</code>	5
<code>markpublish build</code>	5
<code>markpublish templates</code>	6
<code>markpublish export-template</code>	6
<code>markpublish labels</code>	6

KAPITEL 3

Die CLI-Befehle

Alle Befehle im Detail: build, init, cheatsheet, manual, templates, export-template und labels.

Die CLI-Befehle

`markpublish` bietet eine schlanke, intuitive Befehlszeilenschnittstelle (CLI), die über `markpublish` oder die Kurzform `mpub` aufgerufen werden kann.

Befehlsübersicht

Befehl	Kurzbeschreibung
<code>markpublish init</code>	Erzeugt einen minimalen Projektstumpf (zwei Dateien)
<code>markpublish cheatsheet</code>	Rendert die zweiseitige Kurzreferenz
<code>markpublish manual</code>	Rendert dieses Handbuch
<code>markpublish build</code>	Kompiliert das Dokument zu PDF und/oder HTML
<code>markpublish templates</code>	Listet alle gefundenen Templates und deren Quellen auf
<code>markpublish export-template</code>	Exportiert ein Template zur individuellen Anpassung
<code>markpublish labels</code>	Zeigt die aufgelösten statischen Texte und ihre Herkunft

`markpublish init`

Legt einen minimalen Projektstumpf an: eine `markpublish.yaml` und ein Kapitel `next-steps.md`, beide direkt im Zielordner. Kein `chapters/`-Verzeichnis, keine Beispielkapitel.

```
markpublish init [ZIELORDNER] [--title "Titel"]
```

Der Stumpf ist absichtlich klein, weil er dazu da ist, gelöscht zu werden — spätestens beim ersten echten Kapitel. Die Referenz liegt deshalb nicht im Stumpf, sondern in `markpublish cheatsheet`; sowohl die YAML als auch das Kapitel verweisen darauf. Cover und Inhaltsverzeichnis sind abgeschaltet, weil ein einseitiger Entwurf beides nicht braucht.

`markpublish cheatsheet`

Rendert die mitgelieferte Kurzreferenz: Seite 1 die Schlüssel der `markpublish.yaml`, Seite 2 die Grundlagen zu Themes und Templates.

```
markpublish cheatsheet [--lang CODE] [--target pdf|html|all] [--output PFAD] [--theme NAME]
```

markpublish manual

Rendert dieses Handbuch.

```
markpublish manual [--lang CODE] [--target pdf|html|all] [--output PFAD] [--theme NAME]
```

Gemeinsames Verhalten der beiden Dokumentbefehle

Beide Quellen liegen im Paket und werden bei jedem Aufruf neu gerendert. Damit passt das Ergebnis immer zur installierten Version, und ein erfolgreicher Lauf ist zugleich der Nachweis, dass die Rendering-Kette funktioniert — unter Windows also, dass WeasyPrint seine GTK-Laufzeit findet. Geschrieben wird nur das fertige Dokument, und zwar ins aktuelle Verzeichnis; Quelldateien landen nie im Projekt des Nutzers.

Option	Standard	Bedeutung
<code>--lang</code> , <code>-l</code>	Systemsprache	Welche Übersetzung gerendert wird. <code>markpublish manual --lang de</code>
<code>--target</code> , <code>-t</code>	<code>pdf</code>	<code>pdf</code> , <code>html</code> oder <code>all</code>
<code>--output</code> , <code>-o</code>	aktuelles Verzeichnis	Datei oder Verzeichnis
<code>--theme</code>	mitgeliefert	Rendert in einem eigenen Theme

`--lang` wählt die **Quelle**, nicht bloß die Beschriftungen — jede Übersetzung bringt ihre eigene `language:` mit und zieht damit die passenden statischen Texte von selbst nach.

Ohne Angabe entscheidet die Sprache Ihres Systems. Diese beiden Dokumente richten sich an die Person vor dem Rechner und nicht an ein Publikum; deren Sprache ist damit die beste verfügbare Vermutung. Gibt es dafür keine Übersetzung, erscheint kommentarlos die englische — verlangt wurde ja nichts. Ein ausdrückliches `--lang fr` dagegen wird gemeldet, statt still einen englischen Rahmen um eine französische Erwartung zu setzen.

Ohne `--theme` oder `--templates-dir` ignorieren beide Befehle bewusst User- und Projekt-Themes und rendern im mitgelieferten. Sonst zöge ein `templates/` im Arbeitsverzeichnis — mit `export-template` angelegt und noch mitten in der Anpassung — die Referenz mit sich: ein fehlendes Label ließe den Build abbrechen, ausgerechnet in dem Moment, in dem jemand nachschlagen will, wie Labels funktionieren.

Fällt der PDF-Weg mangels GTK aus, liefert `--target html` dieselben Dokumente ohne WeasyPrint.

markpublish build

Kompiliert eine `markpublish.yaml` -Konfiguration.

```
markpublish build [CONFIG_FILE] [OPTIONEN]
```

Argumente & Optionen:

- `CONFIG_FILE` (optional, Standard: `markpublish.yaml`): Pfad zur YAML-Datei.
- `--target / -t` (Standard: `pdf`): Zielformat (`pdf` , `html` oder `all`).
- `--output / -o`: Benutzerdefinierter Ausgabepfad (Datei oder Verzeichnis).
- `--templates-dir`: Spezifischer Pfad zu einem gemeinsamen Template-Verzeichnis.

Beispiele:

```
# Standard-Build aus aktuellem Verzeichnis
markpublish build

# HTML-Version in ein bestimmtes Zielverzeichnis ausgeben
markpublish build markpublish.yaml -t html -o dist/

# Benutzerdefiniertes Template-Verzeichnis nutzen
markpublish build --templates-dir /shared/company-templates
```

Hinweis

`build` hat bewusst kein `--lang`. Die Sprache eines Dokuments gehört in seine eigene `markpublish.yaml`, neben `title` und `author`: sie beschreibt, in welcher Sprache das Dokument geschrieben ist. Ein Override von der Kommandozeile würde lediglich die siebzehn statischen Beschriftungen um unveränderten Fließtext herum austauschen.

`markpublish templates`

Zeigt eine tabellarische Übersicht aller Templates auf dem System und deren Prioritätsstatus:

```
markpublish templates
markpublish templates --target pdf
```

`markpublish export-template`

Kopiert das eingebaute Standard-Template in Ihr lokales Arbeitsverzeichnis:

```
markpublish export-template default templates
```

`markpublish labels`

Zeigt, welcher statische Text am Ende gilt und aus welcher Ebene der Label-Kaskade er stammt. Nützlich, sobald ein Theme oder ein Dokument eigene Texte mitbringt und nicht mehr offensichtlich ist, welche Ebene gewinnt.

```
markpublish labels                # alle Schlüssel, Zielformat PDF
markpublish labels --target html  # Kaskade für die HTML-Ausgabe
markpublish labels --overridden   # nur überschriebene Texte
markpublish labels pfad/zu/markpublish.yaml
```

Argumente & Optionen

Parameter	Typ	Standard	Beschreibung
<code>config_file</code>	Pfad	<code>markpublish.yaml</code>	Pfad zur Konfigurationsdatei
<code>--target</code> , <code>-t</code>	String	<code>pdf</code>	Zielformat, dessen Kaskade gezeigt wird (<code>pdf</code> oder <code>html</code>)
<code>--templates-dir</code>	Pfad	<code>-</code>	Alternativer Template-Ordner
<code>--overridden</code>	Flag	<code>aus</code>	Nur Texte anzeigen, die das Theme ändert

Die Spalte Source nennt die Ebene: `i18n.yaml (de)` für den Programmstandard oder den Pfad einer Theme- bzw. Zielformat- `i18n.yaml` . Unter der Tabelle steht, in welchen Dateien nach Overrides gesucht wurde und welche existieren.

Tipp

`--overridden` beantwortet die häufigste Frage direkt: Was weicht in diesem Projekt überhaupt vom Standard ab?